

```

#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <sstream>

using namespace std;

/*
-----
Email Class
Represents a single email.
-----
*/
class Email
{
private:
    string senderCategory;
    string subject;
    string date;

public:
    Email() {}

    Email(string sender, string subj, string d)
    {
        senderCategory = sender;
        subject = subj;
        date = d;
    }

    string getSender() const
    {
        return senderCategory;
    }

    string getSubject() const
    {
        return subject;
    }

    string getDate() const
    {
        return date;
    }

    int getPriority() const
    {
        if(senderCategory == "Boss")
            return 5;
        if(senderCategory == "Subordinate")
            return 4;
        if(senderCategory == "Peer")
            return 3;
    }

```

```

        if(senderCategory == "ImportantPerson")
            return 2;
        return 1;
    }

    int dateValue() const
    {
        string month = date.substr(0,2);
        string day = date.substr(3,2);
        string year = date.substr(6,4);

        return stoi(year) * 10000 +
            stoi(month) * 100 +
            stoi(day);
    }

    bool operator>(const Email& other) const
    {
        if(getPriority() != other.getPriority())
            return getPriority() > other.getPriority();

        return dateValue() > other.dateValue();
    }
};

/*
-----
MaxHeap Class
List-based implementation using vector.
-----
*/
class MaxHeap
{
private:
    vector<Email> heap;

    void heapifyUp(int index)
    {
        while(index > 0)
        {
            int parent = (index - 1) / 2;

            if(heap[index] > heap[parent])
            {
                swap(heap[index], heap[parent]);
                index = parent;
            }
            else
            {
                break;
            }
        }
    }
};

```

```

void heapifyDown(int index)
{
    int size = heap.size();

    while(true)
    {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;

        if(left < size &&
            heap[left] > heap[largest])
        {
            largest = left;
        }

        if(right < size &&
            heap[right] > heap[largest])
        {
            largest = right;
        }

        if(largest != index)
        {
            swap(heap[index], heap[largest]);
            index = largest;
        }
        else
        {
            break;
        }
    }
}

public:
    void insert(Email email)
    {
        heap.push_back(email);
        heapifyUp(heap.size() - 1);
    }

    void removeMax()
    {
        if(heap.empty())
            return;

        heap[0] = heap.back();
        heap.pop_back();

        if(!heap.empty())
            heapifyDown(0);
    }

    Email getMax() const

```

```

    {
        if(heap.empty())
            return Email();

        return heap[0];
    }

    bool empty() const
    {
        return heap.empty();
    }

    int size() const
    {
        return heap.size();
    }
};

/*
-----
CEOInbox Class
Handles EMAIL, NEXT, READ and COUNT commands.
-----
*/
class CEOInbox
{
private:
    MaxHeap heap;

public:
    void addEmail(string sender,
                  string subject,
                  string date)
    {
        Email e(sender, subject, date);
        heap.insert(e);
    }

    void nextEmail()
    {
        if(heap.empty())
        {
            cout << "No emails available.\n";
            return;
        }

        Email e = heap.getMax();

        cout << "\nNext email:\n";
        cout << "Sender: "
              << e.getSender() << endl;
        cout << "Subject: "
              << e.getSubject() << endl;
        cout << "Date: "

```

```

        << e.getDate() << endl;
    }

    void readEmail()
    {
        heap.removeMax();
    }

    void countEmails()
    {
        cout << "There are "
              << heap.size()
              << " emails to read."
              << endl;
    }
};

/*
-----
Main Program
Reads commands from input.txt.
-----
*/
int main()
{
    CEOInbox inbox;

    ifstream file("input.txt");

    string line;

    while(getline(file, line))
    {
        if(line.rfind("EMAIL ", 0) == 0)
        {
            string data = line.substr(6);

            stringstream ss(data);

            string sender;
            string subject;
            string date;

            getline(ss, sender, ',');
            getline(ss, subject, ',');
            getline(ss, date);

            inbox.addEmail(sender,
                           subject,
                           date);
        }
        else if(line == "COUNT")
        {
            inbox.countEmails();
        }
    }
}

```

```
    }
    else if(line == "NEXT")
    {
        inbox.nextEmail();
    }
    else if(line == "READ")
    {
        inbox.readEmail();
    }
}

file.close();

return 0;
}
```